Future Generation Computer Systems 00 (2027) 1–6

Who should look at this image? Small language agents choosing the tools of inspection drones

Mahdi Khashan*

[a][Department, Institution], [City], [Country]

Abstract

Inspection drones can run small vision models on board and send images to larger models in the cloud. Choosing a model for each image is a tool-selection problem with a time budget and a weak radio link. Recent work trains small language models to select tools by supervised fine-tuning and then by reinforcement learning. We test this approach on power-line inspection. A text-only language model with 0.6 billion parameters chooses, step by step, among about 60 vision tools and then writes a report with three parts: component type, whether there is a defect, and condition. We evaluate the reports against the true labels of 1,500 test images, with recorded mobile-network traces and time budgets of 4 to 30 seconds. After supervised fine-tuning, the agent learns to call one large general model and stop: 88.1% of its reports are fully correct, at 0.35 seconds and 34 kilobytes of upload per image. When this model is removed and a small component classifier is added, the same recipe produces an agent that combines two small onboard classifiers: 95.9% of its reports are fully correct, at 0.09 seconds per image and with no upload. Reinforcement learning after fine-tuning changes nothing measurable in either case, because the fine-tuned model is so certain that its sampled answers almost always receive the same reward. The tool catalogue therefore matters more than the learning method. Step-wise training targets also favour cheap tools that are usually right over the most accurate ones.

Keywords: drone inspection, edge intelligence, tool selection, small language models, reinforcement learning

1. Introduction

Drones are changing how power lines, bridges and towers are inspected. They take many high-resolution images during a flight, and new drones can also analyse these images on board [1]. A drone can use many AI models, which we call *tools*: small classifiers on the drone, vision-language models (VLMs) on the drone, and large models in the cloud that need an image upload over a radio link that is often slow or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson board and need 1.4–6.4 s for each video segment [2]. Azimi et al. compare running such a model on the drone, in the cloud, or split between the two [3]. Both groups name an adaptive controller, which decides at run time which tool to use, as future work.

Language models that use tools are now common [8, 9, 10]. Zhou et al. [5] show that even small language models can learn to select tools from a large catalogue when they are trained with supervised fine-tuning (SFT) and then with group relative policy optimisation (GRPO) [6]. A small model of this kind could run next to the drone's other software and plan the inspection of each image.

This article asks a simple question: *can a small language model learn to choose the tools of an inspection drone, and does reinforcement learning improve on imitation?* We build a catalogue of about 60 vision tools, cache their answers on every image, and let a text-only Qwen3-0.6B [7] agent call tools step by step until it writes a report. We train it with the recipe of Zhou et al. and compare it with the same model without training and with each tool alone.

Our main contributions are:

- a tool-selection benchmark for drone inspection with 62 real vision tools, measured latency, mobile-network traces and an upload cost that is paid once per image;
- a test of the SFT and GRPO recipe for small tool-selecting language models in this setting, evaluated by report accuracy against true labels;
- evidence that the tool catalogue matters more than the learning method: replacing one large generalist model by a small specialist raises the share of fully correct reports from 88.1% to 95.9% and removes the upload;
- an analysis of why GRPO adds nothing after SFT here, and why the agent ignores the most accurate tools;
- a thinking variant trained on reasons written by a larger teacher model (results in progress).

Section 2 reviews related work. Section 3 describes the data, the tools, the agent and its training. Section 4 reports and discusses the results, and Section 5 concludes.

2. Related Work

Edge VLMs for inspection. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson and reduces the upload of a 4 s

*Corresponding author.
Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

video segment from 485 kB to 25 kB. The study in [3] measures SmolVLM on the drone, in the cloud, and split between them, under different bandwidths. INAR-VL [4] sends requests to the edge or the cloud based on the input and the bandwidth. Farahani et al. [1] show that SmolVLM-256 needs about 2.8 s per image on a Jetson Orin, while a YOLO detector needs 0.14 s, and they conclude that a light model should trigger the VLM only when needed. These systems use fixed rules to choose where one model runs; they do not choose, per image, among many models with different strengths, costs and locations.

Tool-using language models. Toolformer [8] and ReAct [9] let a language model call tools inside its own text. ToolLLM [10] and Gorilla [11] train models on thousands of APIs and use a retriever to shorten the list of candidates. Zhou et al. [5] split the problem into planning, retrieval and selection and show that small models trained with SFT and GRPO transfer to new tools. Distilling the reasoning of a larger model into a small one improves small models on several tasks [12]. These benchmarks use software APIs without time limits, networks or a real cost per call; the tools rarely disagree, and success is usually a correct call rather than a correct final answer.

Our earlier work. On the same data, a learned policy (PPO [15]) that chose among four tools matched but did not beat well-tuned rules, and combining the tools' answers with a fixed formula gave the largest gain. Here the language model itself reads the tool answers and writes the report, so no fusion formula is used.

3. Methodology

3.1. Data

We use component images of InsPLAD [23] with the `tower14` split of ElecVQA-Bench [22]: 18 training towers for the tools, 10 validation towers (5,624 images) for the agent's training data, and 14 test towers (6,092 images). There are 11 component types and six conditions: good, rust, corrosion, missing cap, bird nest and other defect. Only 9.9% of the test images show a defect.

3.2. Tools

Table 1 groups the 62 tools. We fine-tuned the classifiers on the training towers and ran every tool once on every validation and test image; episodes replay these cached answers. The three laya-vision tools [18] name the component together with defect and condition. We added a ResNet-18 component classifier later (Section 4.3); the main experiment uses the other 61 tools. Fourteen tools run on the drone (for example ResNet-18, DeiT-S, MobileNet, the component classifier, the 4-bit laya-vision and SmolVLM); the other 48 run in the cloud. We measured the GPU time of each tool on a desktop GPU. Because we have no Jetson, we multiply onboard times by 9, our estimate of the speed gap to a Jetson Orin NX.

3.3. Network and constraints

A cloud call costs its GPU time, a round trip of 50 ms, and the upload of the image, which is paid once per image. The upload follows recorded 4G/LTE traces [24] reduced to 10% of their bandwidth (about 3 Mbit/s), with outages during which no cloud tool can run. A median image has 33 kB. The agent has a time budget of 4, 10 or 30 s per image and at most 8 steps; at the eighth step it must report.

Table 1. The 62 tools in groups. Answer: the fields that the tool can answer (C: component, D: defect, S: condition).

Group	Examples	Tools	Answer
Image classifiers	ResNet-18/50/101 [19], ConvNeXt, ViT, DeiT, Swin, EfficientNet, MobileNet, YOLO	12	D, S
Classifier variants	flip, five crops, re-cut from the full photo	6	D, S
Component specialists	ResNet-50 and ConvNeXt per component group	10	D, S
Feature probes	DINOv2 [20], CLIP [21]: condition classifier and defect detectors	15	D or S
Zero-shot	CLIP-L with text prompts	1	D, S
Small VLMs	SmolVLM-256M [17], 3 variants	3	D
laya-vision	full, 4-bit, re-cut	3	C, D, S
Large VLM	Qwen3-VL-8B, 8 questions or crops	8	D
Component classifier	ResNet-18 on 11 component types	1	C
Image checks	quality, exposure, size	3	–

3.4. The agent

An episode starts with no tool results. At each step the agent receives a prompt with the task, the time left, the link state, whether the image is already uploaded, the answers of the tools called so far as short text, and a list of candidates. A BM25 retriever [13] selects 15 candidates from the tools that still fit in the time, using the task and the results so far as the query; *report* is always added. The agent writes one JSON object: either it selects a tool, whose cached answer is added to the next prompt, or it writes the report. The agent never sees the image, only the tools' text answers.

3.5. Problem formulation

Equations (1)–(6) define the cost of a tool, the tools that can still be called, the correct fields of an answer, the training target, the reward and the GRPO advantage. Table 2 lists the symbols.

$$c(t) = \begin{cases} 9\, g_t & t \text{ on the drone} \\ g_t + t_{\mathrm{rtt}} + \frac{8\, s_i}{10^6\, b}\, \mathbb{1}[i \notin U] & t \text{ in the cloud, } b > 0 \\ \infty & t \text{ in the cloud, } b = 0 \end{cases} \quad (1)$$

$$F = \{t \notin K : \; c(t) < \infty, \; \tau + c(t) \le B\} \quad (2)$$

$$\kappa(a, y) = \{f \in \{\mathrm{C}, \mathrm{D}, \mathrm{S}\} : \; a_f = y_f\} \quad (3)$$

$$t^* = \arg\max_{t \in F} \left(\left| \bigcup_{k \in K \cup \{t\}} \kappa(a_k, y) \right| - \left| \bigcup_{k \in K} \kappa(a_k, y) \right|, \; -c(t) \right) \quad (4)$$

Table 2. Symbols in Eqs. (1)–(6).

Symbol	Meaning
g_t	GPU time of tool t on a desktop GPU
t_{rtt}, s_i, b	round trip (50 ms); size of image i in bytes; bandwidth in Mbit/s
U	images already uploaded in this episode
K, τ, B	tools called so far; time used; time budget (4, 10, 30 s)
a_k, y	answer of tool k (only the fields it gives); true labels
t^*	training target: the tool that adds most correct fields (ties: faster); report if the gain is zero
o_j, G	answer j of a group of $G = 8$ sampled answers

$$R(o) = \begin{cases} \mathbb{1}[o = t^*] & o \text{ selects a tool} \\ \frac{1}{3}|\kappa(o,y)|\,\mathbb{1}[t^* = \textsc{report}] & o \text{ is a report} \\ 0 & o \text{ not valid} \end{cases} \tag{5}$$

$$\hat{A}_j = \frac{R(o_j) - \text{mean}_k\, R(o_k)}{\text{std}_k\, R(o_k)} \tag{6}$$

3.6. Training

Following Zhou et al. [5], we train each step separately. For a tool step, the target is t^* of Eq. (4). For a report step, the target is the true labels, so the agent must learn whom to trust when tools disagree. We built 61,461 training steps from 22,496 episodes (validation images × 4 budgets, with 30% random tool steps for variety).

SFT trains LoRA [16] adapters (rank 16, all linear layers) on 12,000 steps for one epoch with TRL [14]. GRPO then continues from the SFT adapter for 400 steps on 4,000 prompts. It samples 8 answers per prompt and rewards each answer as in Eq. (5). The *base* model is the same Qwen3-0.6B with the same instructions and no training.

Thinking variant. After SFT, the 8 sampled answers are often identical, so GRPO receives no learning signal (Section 4.4). We therefore let a larger model, Claude Haiku 5.5, write a short reason (80–150 words) for 4,000 balanced steps. The reason explains the state, the missing fields, the time and the link, and why the target action fits, without referring to the label. 3,780 reasons passed automatic checks. SFT then trains the agent to write a reason before each action, and GRPO continues with up to 384 new tokens per step. The reasons of earlier steps are not kept in later prompts.

4. Experimental Evaluation

We test on 1,500 of the 6,092 test images, drawn with a fixed seed, at each budget. The metric is the share of reports in which each field, and all three fields, match the true labels. All runs use one seed and greedy decoding.

4.1. The tools alone

Table 3 shows the accuracy of some tools when each one answers alone. Fine-tuned classifiers are strong on defect and condition; a ConvNeXt on the re-cut component is correct on 98.8% of the images. In the main experiment, laya-vision is the only tool that names the component, and it is correct on 92.4% of the images; the component classifier added in Section 4.3 is correct on 99.6%. If the agent could always pick a correct tool among the 61 tools of the main experiment, it would reach 97.0% on the component and 100% on defect and condition.

Table 3. Accuracy of single tools on all 6,092 test images. "–": the tool gives no answer for this field. Last row: share of images where at least one of the 61 tools of the main experiment is correct.

Tool	Runs on	Component	Defect	Condition
laya-vision	cloud	0.924	0.947	0.952
laya-vision, 4-bit	drone	0.870	0.944	0.938
ResNet-18	drone	–	0.981	0.979
DeiT-S	drone	–	0.982	0.980
ConvNeXt-T, re-cut	cloud	–	**0.988**	**0.988**
Component ResNet-18	drone	**0.996**	–	–
At least one tool correct		0.970	1.000	1.000

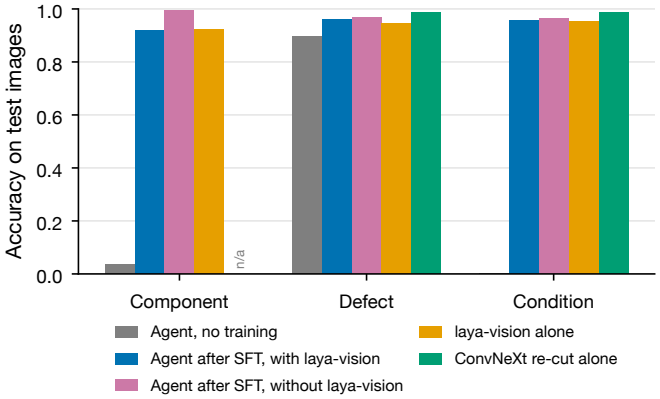


Figure 1. **The tool catalogue decides what the agent can reach.** Accuracy per report field at the 4 s budget. With laya-vision, the trained agent matches laya-vision; without it, the agent combines two drone classifiers and is better on every field. Agents: 1,500 test images; single tools: all 6,092 test images; single seed. SFT and SFT + GRPO give identical results. ConvNeXt re-cut gives no component (n/a).

4.2. SFT teaches the format and one strong tool

Table 4 and Figure 1 give the report accuracy of the agent. Without training, Qwen3-0.6B never calls a tool and writes a report at the first step. Its component and condition answers almost never match the label set, and it always says "no defect", so its defect accuracy (0.895) equals the share of good images and its defect recall is zero. After SFT, 88.1% of the reports are fully correct. The agent calls exactly one tool per image: laya-vision in the cloud on 95% of the images and the 4-bit laya-vision on the drone on the others, mainly when the link is down. Then it reports.

The agent is slightly better than the tool it copies. Its component accuracy equals that of laya-vision (0.92), but its defect and condition answers are slightly better (0.961 against 0.947, and 0.958 against 0.952). It sees only laya-vision's answer, so it has learned to correct some of laya-vision's answers when it writes the report.

The agent ignores the best tools. A ConvNeXt on the re-

Table 4. Report accuracy of the Qwen3-0.6B agent on 1,500 test images, single seed, greedy decoding. All three: share of reports with component, defect and condition correct. Time and upload: mean per image. Results without a time limit are left out because of a fault in the cost check (Section 4.7). Best results in bold.

Method	Budget	Component	Defect	Condition	All three	Defect recall	Time	Upload
Base (no training)	4 s	0.037	0.895	0.000	0.000	0.000	0.00 s	0.04 kB
SFT	4 s	**0.919**	**0.961**	**0.958**	**0.881**	0.885	0.35 s	34.1 kB
SFT	10 s	0.917	**0.961**	**0.958**	0.880	**0.892**	0.37 s	34.1 kB
SFT	30 s	0.915	0.959	0.957	0.876	0.885	0.41 s	33.7 kB
SFT + GRPO	4 s	**0.919**	**0.961**	**0.958**	**0.881**	0.885	0.35 s	34.1 kB
SFT + GRPO	10 s	0.917	**0.961**	**0.958**	0.880	**0.892**	0.37 s	34.1 kB
SFT + GRPO	30 s	0.915	0.959	0.957	0.876	0.885	0.41 s	33.7 kB

cut image is 2.7 points better on defect and 3.0 points better on condition than the agent, and DeiT-S on the drone is 2 points better. The agent never calls them. The reason is the training target in Eq. (4): laya-vision adds up to three correct fields at once, so it is the target in almost every first step (24,577 of the 30,950 tool steps). A second tool is the target only when laya-vision is wrong, which is rare, and the report step then sees only laya-vision's answer.

The budgets have no effect. One laya-vision call costs about 0.35 s, far below the smallest budget of 4 s, so the agent behaves the same at every budget.

4.3. A small specialist instead of a large generalist

Since laya-vision is the only tool that names the component, the agent cannot answer all three questions without it. We therefore trained a ResNet-18 on the 11 component types of the training towers and added it as a drone tool that answers only the component. It is correct on 99.6% of the test images (Table 3) and costs 2.8 ms of GPU time. We then removed the three laya-vision tools, rebuilt the training data with the same target rule, and trained the agent with the same recipe.

Table 5 shows the result. The agent now calls two tools on every image: the component classifier, and ResNet-50 (95%) or ResNet-18 (5%) for defect and condition. All three run on the drone, so nothing is uploaded. The share of fully correct reports rises from 88.1% to 95.9%, and the time per image falls from 0.35 s to 0.09 s. Most of the gain comes from the component; defect and condition improve by less than one point, and the defect recall stays the same. GRPO again changes no answer: 95–99% of its groups had no spread in reward.

The agent still does not reach the best tools: ConvNeXt on the re-cut image would be about two points better on defect and condition. The target rule picks the fastest tool that is correct on each training image, and ResNet-50 is correct on 97% of them, so ConvNeXt is the target in only 12 of about 84,600 training steps.

4.4. Why GRPO does not help

GRPO learns only from groups whose sampled answers receive different rewards (Eq. (6)). Table 6 shows that after SFT, 91–95% of the groups had no spread in reward. The model was almost certain (entropy about 0.002), so the eight samples were usually the same answer. The adapter weights moved by at most 0.001, and the KL distance to the SFT model stayed near 0.002. No most-likely answer changed, so the evaluation results are identical to SFT in every digit.

With teacher reasoning, the eight answers differ: only 62–71% of the groups had no spread, and the model moved about 30 times further from its SFT starting point. The training reward did not increase, however (0.845 in the first half, 0.811 in the second), so it is not yet clear that the extra signal points in a useful direction.

4.5. Thinking and larger models

Table 7 will compare the thinking variant at three model sizes. SFT on the teacher reasons converged to a training loss of about 0.89 (0.6B) and 0.81 (1.7B); the 4B model is trained in bf16 on an A10 GPU. The evaluations are running.

4.6. Latency and upload

Table 8 compares the time and upload of the tools that the agent calls with published Jetson measurements. The units differ: an EVLM segment contains several frames and a text report, while our item is one image with three answers. The agent's own inference time is not included.

4.7. Discussion

The catalogue matters more than the learner. The same recipe gave 7.8 more points of fully correct reports when one generalist model was replaced by a small specialist, while GRPO changed nothing with either catalogue. Designing and measuring the tools is at least as important as training the agent.

Imitation sets the ceiling. SFT teaches the agent the target policy almost exactly, and that policy is "call the tool that answers most fields, then report". A target that counts fields favours generalist tools over accurate specialists. Better targets would reward the final report, not the single step.

GRPO needs disagreement. After SFT on short answers, the model is too certain for group-based rewards to differ. Teacher reasoning restores the disagreement, but whether it also improves the reports is the open question of Table 7. Rewarding all tools that are equally good, and training on full episodes with the final report accuracy as reward, are the next steps.

Budgets must bind. With 4 s or more and a 0.35 s first call, the time budget never limits the agent. Tighter budgets, a cost for upload, or slower onboard tools are needed before the agent has to trade accuracy for time.

Table 5. The same recipe with a different tool catalogue. Qwen3-0.6B agent at the 4 s budget, 1,500 test images, single seed. Without laya-vision, a ResNet-18 component classifier is available. Calls: tools called per image. Best results in bold.

Tools	Method	Component	Defect	Condition	All three	Defect recall	Time	Upload	Calls
With laya-vision	SFT	0.919	0.961	0.958	0.881	**0.885**	0.35 s	34.1 kB	1.0
Without laya-vision	base	0.019	0.346	0.003	0.001	0.452	0.14 s	19.2 kB	0.4
Without laya-vision	SFT	**0.994**	**0.967**	**0.965**	**0.959**	0.879	**0.09 s**	**0 kB**	2.0
Without laya-vision	SFT + GRPO	**0.994**	**0.967**	**0.965**	**0.959**	0.879	**0.09 s**	**0 kB**	2.0

Table 6. GRPO training of the 0.6B agent, mean over the first and second half of training. No signal: share of groups in which all 8 answers got the same reward.

	Without thinking	With thinking
Steps (prompts per step)	400 (2)	200 (1)
Answer length	≈22 tokens	≈150 tokens
No signal, 1st / 2nd half	0.91 / 0.95	0.62 / 0.71
Reward, 1st / 2nd half	0.861 / 0.903	0.845 / 0.811
KL to the SFT model	≈0.002	≈0.06

Table 7. Thinking variant, report accuracy at the 4 s budget. All methods write a reason before each action. Dashes: evaluation in progress.

Model	Method	Comp.	Defect	Cond.	All three
Qwen3-0.6B	base	–	–	–	–
	SFT	–	–	–	–
	SFT + GRPO	–	–	–	–
Qwen3-1.7B	base	–	–	–	–
	SFT	–	–	–	–
	SFT + GRPO	–	–	–	–
Qwen3-4B	base	–	–	–	–
	SFT	–	–	–	–

Table 8. Latency and upload. Prior work: measured on a Jetson Orin. This work: mean per test image at the 4 s budget, tool time only.

System	Time	Upload
Prior work		
YOLO detector, per image [1]	0.14 s	–
SmolVLM-256, per image [1]	2.8 s	–
EVLM (SmolVLM), per 4 s segment [2]	1.4–6.4 s	25 kB
SmolVLM on the drone, per request [3]	2.7–2.8 s	0
This work		
SFT agent (one laya-vision call)	0.35 s	34.1 kB
SFT agent without laya-vision (two drone classifiers)	0.09 s	0
ResNet-18 on the drone, alone	0.03 s	0

Limitations. The tool answers are cached and replayed; the network and the onboard times are simulated, and the factor of 9 is an estimate. The time of the agent itself is not counted. A fault in the cost check allowed cloud tools during outages when there was no time limit; it affects 1.1% of the training steps and all results without a limit, which we leave out. All results come from one seed and one benchmark of power-line components.

5. Conclusion

A small language model can learn to inspect power-line images by calling tools. After SFT, a Qwen3-0.6B agent writes a fully correct report for 88% of the images when a large general model is available, and it learns to call that model and stop. When the general model is replaced by a small component classifier, the same recipe produces an agent that combines two onboard classifiers and writes a fully correct report for 96% of the images in 0.09 s, with no upload. GRPO after SFT adds nothing with either tool catalogue, because the model's sampled answers do not differ. For drone inspection, the choice of tools matters more than the learning method, and the recipe of small tool-selecting models needs training targets that reward the final report rather than the cheapest correct step. Teacher reasoning and larger models are under evaluation. Future work will measure the agent and the onboard tools on a Jetson, train on full episodes, and test tighter budgets.

Acknowledgements

Experiments in this work were run and tracked with OpenResearch [25].

Data and code availability

The code, the cached tool answers and the result files are in the project repository, branch `agentic-ppo-v2` (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [22, 23]. The laya-vision weights are published under CC BY-NC-SA 4.0 [18].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text, and a language model to write the teacher reasons used as training data. The author reviewed and edited the content and takes full responsibility for the publication.

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[5] Zhou et al., Training small language models for transferable tool selection, arXiv:2610.31483 (2026).

[6] Z. Shao, et al., DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, arXiv:2402.03300 (2024).

[7] A. Yang, et al., Qwen3 technical report, arXiv:2505.09388 (2025).

[8] T. Schick, et al., Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[9] S. Yao, et al., ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[10] Y. Qin, et al., ToolLLM: Facilitating large language models to master 16000+ real-world APIs, in: International Conference on Learning Representations (ICLR), 2024.

[11] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive APIs, arXiv:2305.15334 (2023).

[12] C.-Y. Hsieh, et al., Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes, in: Findings of the Association for Computational Linguistics (ACL), 2023.

[13] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.

[14] L. von Werra, et al., TRL: Transformer reinforcement learning, software (2020). https://github.com/huggingface/trl.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).

[16] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[17] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[18] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[20] M. Oquab, et al., DINOv2: Learning robust visual features without supervision, Transactions on Machine Learning Research (2024).

[21] A. Radford, et al., Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning (ICML), 2021.

[22] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[23] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[24] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.

[25] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.